2024 IEEE 48th Annual Computers, Software, and Applications Conference (COMPSAC) | 979-8-3503-7696-8/24/$31.00 ©2024 IEEE | DOI: 10.1109/COMPSAC61105.2024.00015

Test Case Selection in Continuous Regression Testing using Machine Learning: An Industrial Case Study

Azeem Ahmad
Ericsson AB, Linköping University
Linköping, Sweden
azeem.ahmad@(ericsson.com,liu.se)

Dimistris Rentas
Ericsson AB
Linköping, Sweden
dimitris.rentas@ericsson.com

Daniel Hasselqvist
Ericsson AB
Linköping, Sweden
daniel.hasselqvist@ericsson.com

Pontus Sandberg
Ericsson AB
Linköping, Sweden
pontus.sandberg@ericsson.com

Kristian Sandahl
Linköping University
Linköping, Sweden
kristian.sandahl@liu.se

Aneta Vulgarakis
Ericsson AB
Linköping, Sweden
aneta.vulgarakis@ericsson.com

***Abstract*—Continuous integration and delivery (CI/CD) have transformed software development by reducing delivery time, improving product quality, and giving enterprises a competitive advantage. However, large-scale projects confront difficulties in giving fast feedback to developers due to large test suites, resulting in longer testing cycles and lower productivity. Traditional regression testing methods struggle to find a balance between efficacy and efficiency, demanding advanced approaches. This study investigates the use of machine learning (ML), specifically Neural Network and Random Forest models, to choose test cases based on source code changes, commit messages, and change file path in order to offer developers with faster feedback. The study investigates the predicted accuracy of ML models using a large industrial dataset from a telecom company, which included 15 million test executions over 15 months. The results show that Random Forest outperforms Neural Network models in test case selection, with up to 97% accuracy achieved. Real-time evaluations conducted over a month show significant savings in test executions (88%-90%) and testing time (44%-74%) across multiple regression testing activities, illustrating the potential of ML-driven techniques to optimize CI/CD pipelines and increase developer productivity.**

***Index Terms*—Test Case Selection, Machine Learning Models, Test Case Selection on Code Changes, Multi-factor Test Case Selection**

I. Introduction

Organizations have benefited greatly [1]–[5] from continuous integration and delivery (CI/CD), which has accelerated delivery time to market, enhanced product quality, and given companies a competitive edge. With CI/CD in place, large-scale organizations find it difficult to give developers faster feedback related to test executions on frequent source code changes. Complex and large-scale products consists of large test suites, thus increasing time for feedback leading to potential risk to achieve faster delivery. Developer productivity decreases as test suites grow, and execution time becomes prohibitive due to slower feedback from testing cycles [6]–[8] and an overwhelming amount of test artifacts to analyze [7]. Developers at Google wait between 45 minutes to 9 hours to receive feedback from test executions [6]. Shahin et al. [9] discovered that CI processes begin to fail when developers are unable to acquire feedback from tests in a timely manner. Another study [10] reports that even with high-performance machines, Microsoft's regression tests for a single software package take several days. Similarly, for its 13K projects, Google's Test Automation Platform required 150 million tests [6]. Thus, regression testing becomes a critical bottleneck in the pipeline increasing feedback cycles related to the quality of the developed product.

How to select a subset of test cases during regression testing is a well-known problem requiring faster feedback to developers. It is not suitable to run all test cases on each commit or modification in software under test (i.e., SUT). Running all test cases on each commit or change is resource exhaustive and not optimized. The state-of-art test case selection techniques included black and white box methodologies, and a lot has been studied in the study [11]. Such techniques are based on Historical data [12], requirement coverage [13], historical ability to detect faults [14], similarity between test cases [15], and customized approaches [16]. Still, Regression testing is a demanding undertaking due to the trade-off between efficacy and efficiency, as well as difficulty in capturing interconnections in complex software systems, requiring more intelligent strategies [17]. Recent breakthroughs in AI/ML have enabled researchers and practitioners to examine smart strategies for selecting test cases more effectively. Such machine learning algorithms go beyond static relationships and use patterns to select test cases as input data (i.e., historical information, new faults, commit messages etc.,) changes.

In this paper, we have used machine learning approaches (i.e., Neural Network and Random Forest) on large industrial

2836-3795/24/$31.00 ©2024 IEEE
DOI 10.1109/COMPSAC61105.2024.00015

data set (i.e., telecommunication company) to select test cases based on source code changes, commit messages and changed file paths. We call the aforementioned attributes as an input factors. The paper presents the effect of each input factor as well factors' combinations on models' test case selection. A big data set containing 15 million test executions (i.e., single test executions) consist of commit messages, changed source code, and changed file paths was collected from the CI pipeline. The data spans 15 months.

In addition to training and testing on collected data, the models were assessed using real-time evaluation over a month to calculate the time and execution savings if the predicted test cases had been executed as compared to what has actually been executed by the developers. This study attempts to answer the following questions:

RQ1: What is the predicted accuracy of machine learning models when selecting test cases based on code changes, commit messages, and changed file paths?

[RQ1.1:] How does each input factor affect the machine learning model's predictions/selection? Are there more effective combinations of factors, or are some factors better in isolation?

RQ2: What are the advantages of running model-predicting test cases compared to running tests selected by humans?

II. Research Methods

A. Case

This case study was performed with data collected from telecommunication company located in Sweden. We perform an exploratory case study with the objective of exploring the benefits of using machine learning based test case selection to providing fast and meaningful feedback on integration test activities. Two testing activities were investigated in this study: Pre-merge testing activities and CI loop test activities. Pre-merge activities are early phase tests in which developers run test cases in a local CI pipeline to ensure that their code changes are of high quality. Running all test cases on a single change increases the time required to receive early feedback on the changes. Some of the developers may have gained a knowledge of which test cases to execute based on code changes over time; however, freshly hired developers lack a thorough understanding of the test cases or SUT, so they execute all test cases to be safe, causing servers to overload and queue time to increase. Given the large number of test cases, we would like to provide developers with a system that allows for automated selection of test cases based on code changes, minimizing time to feedback.

Once the developers have established confidence in their code changes on the local CI testing pipeline (pre-merge activities), they merge the commit into the CI loop (another testing activity for our scope). All test cases are executed again on each merged commit. There is frequent code merging in a day, therefore identical test executions happen a lot in the CI Loop in addition to nightly testing, where the test cases are executed the third time for regression testing. The aim is to provide a faster feedback to developers from pre-merge activities, CI loop and nightly runs. The test cases and SUT are written in C++, and the process of developing test cases is confidential not related to our work because the test case selection is based on source code changes and does not take the test case code into account.

B. Data Collection, Pre-Processing and Analysis

The test execution data was collected from CI pipeline between January 2022 and March 2023 consisted of approximately 11 Million test executions. The 11 millions does not represent the unique test executions. For example, one test suite execution contains *X* number of test cases with commit that modified two source files (i.e., *f1 and f2*), leading to total *X*2* number of recorded observations in the collected data. The aim is to record all test execution separately with each modified file (i.e., *f1 and f2*). Due to confidentiality, we cannot share unique test executions. From the non-unique test executions data frame, we can share that passed test cases were 63.4% and failed test cases were 36.4%. The rest of test executions contains verdict labeling (i.e., skip, flaky, not applicable etc.,) that is out of scope of this paper. The dataset was divided into training set and test set which contained 70% and 30% of the whole data respectively. The collected data consists of *'Test Suite Name'*, *'Test Case Name'*, *'Verdict'*, *'Commit Message'*, *'Changed File Path'*, *'Average Test Time'*, *'Modified Source Code (i.e., only addition)'*, and *'Build ID'*, as shown in Figure 1. There are 4 features and 1 label (i.e, verdict) in the data set: name of the test cases, commit message, changed file path, and added code. Although these features are in text format, different approaches were used to transform each of the features due to their different properties. The label is thus the verdict of the test case which is binary (i.e., pass or fail).

Figure 1 presents a detailed method for data collection, parsing, analyzing and ML model development and evaluation. For pre-processing, as shown in Figure 1, stop words were filtered using a specified list of phrases and later, the commit messages were divided into tokens, with some tweaks made to filter special characters and punctuation using regex. Following the initial processing, the messages were processed using Term frequency-inverse document frequency (TF-IDF) to vectorize commit messages into bags of words (TF) and assess the frequency of each word (IDF). The functions used for this were "CountVectorizer" and "IDF".

The detailed information about how a single commit message was pre-processed inspired by strategies adopted by Yang et al. [18]. For processing the added source code, the approach was inspired by Yang et al. [18] and Khalid et al. [19]. The regex matches a specific set of characters within added code including whitespace, tab, brackets (curly and square brackets), parentheses, and special characters (like arithmetic operators). The resulting tokens were the code itself, which

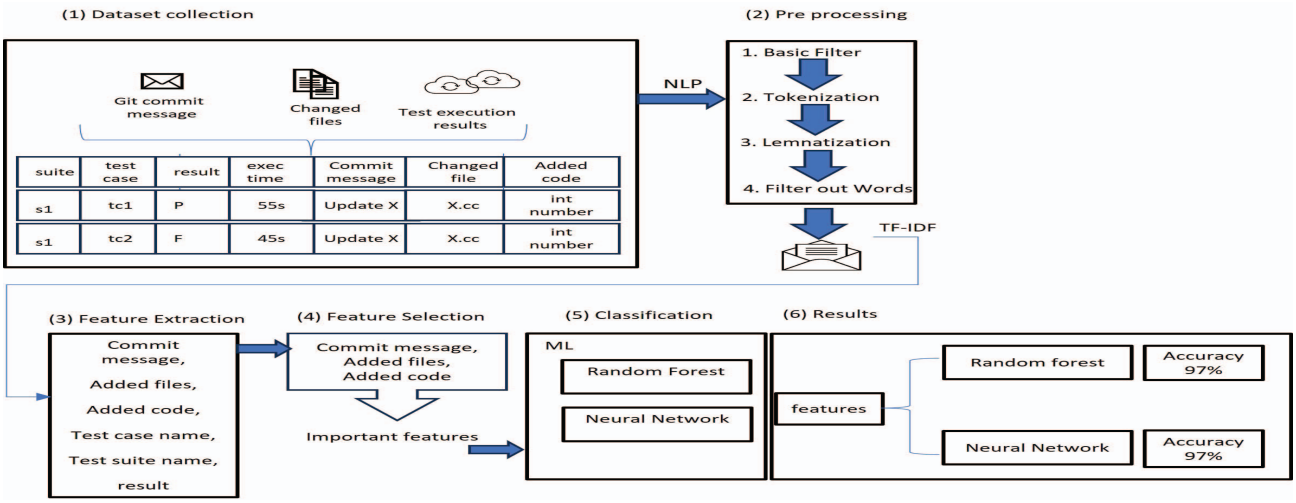

Fig. 1. A Methodology Overview Inspired by Yang et al. [18]

was then transformed using TF-IDF. The experiment is carried out on experimentally allocated distributed cloud clusters that are shared internally by all employees. The cluster itself provides adequate computing power and memory, but because it is shared by a large number of employees, performance may vary. PySpark was used to allocate jobs to different nodes, and the model was implemented using Python and well-known machine learning packages.

C. Machine Learning Models, Parameter Tuning & Matrices

In our research, we use both Random Forest (RF) and Neural Networks (NN) to address the complex nature of our dataset. Random Forest excels at handling high-dimensional data, noisy features, and big datasets, delivering solid predictions via ensemble learning and decision tree averaging. Its transparency and interpretability make it especially useful when the model's decision-making process must be easily understood. Furthermore, RF requires low preprocessing and parameter adjustment, making it ideal for exploratory research and early model building. While neural networks provide incredible flexibility and the ability to catch nuanced patterns within data, their black-box nature often makes comprehending model decisions difficult, particularly in sectors where openness is critical.

A decision tree is a supervised learning technique used for classification and regression, with the purpose of predicting the value of a target variable using one or more input variables. Random forest is an ensemble learning method that mixes several decision trees to produce a more accurate and robust model. It is based on the principle of bootstrap aggregation, which involves averaging numerous independent forecasts to lower the variance of a decision tree. We have used different *Depth - Mtry* (i.e., 10, 20 and 30) and different *NTree* (i.e., 25, 50, 100, 200, 300).

Deep neural networks (DNN) are the foundation of many deep learning systems. A DNN consists of several layers, including an input layer, an output layer, and one or more hidden layers. Each layer is composed of a succession of neurons. Weighted edges connect neurons across layers. Each neuron is a computer unit that uses an activation function to process its inputs and the weights of incoming edges. The computed result is transmitted to the next layer via the edges. The weights of the edges are not directly given by software developers, but rather learned automatically during a training process using a huge quantity of labeled training data. Finally, an arrangement of three hidden layers with 10, 20, and 10 neurons was determined to produce the greatest results inspired by the work of Feng et al. [20]

Evaluating classifiers solely on accuracy is insufficient [21]. We need to examine precision, recall, and f1-score [21]. We defined a TP (True positive) case as one in which a given failed testcase is accurately predicted, whereas a TN (True negative) case is one in which a given passed test case is correctly predicted. A FN (False negative) case occurs when a given test case fails but we predicted it to succeed, whereas an FP (False positive) case occurs when a given test case passes but was predicted to fail. Precision is defined as the number of correctly classified failed tests divided by the total number of tests labeled failed. F1 - Determines how well our model handles the trade-off between precision and recall.

Since we would rather misclassify a passed test case than a failed test case, recall becomes the most relevant statistic to look at. However, precision is still significant because we don't have limitless time to execute the testcases.

In addition, we investigate the saving in test time and tests executions, if model predicted test cases were executed as compared to human selected test cases during the live evaluation (Section III-B). We do not show the statistics of failures that occur in the CI pipeline or the average percentage of failure detection (APFD) in test cases that are predicted by the model. Such data is organizationally sensitive. The APFD's absence could raise concerns about fault slippage.

TABLE I
THE COMPARISON IN TERMS OF ACCURACY, PRECISION, RECALL AND F1-SCOPE BETWEEN RANDOM FOREST AND NEURAL NETWORK

	Accuracy		Precision		Recall		F1-Score	
Factors	RF	NN	RF	NN	RF	NN	RF	NN
Commit Message	0,95	0,94	0,97	0,96	0,93	0,95	0,95	0,94
Added Code	0,92	0,87	0,94	0,92	0,91	0,85	0,92	0,87
Change Files	0,97	0,87	0,97	0,88	0,97	0,91	0,97	0,87
Commit Message, Added Code	0,97	0,94	0,97	0,94	0,97	0,95	0,98	0,94
Commit Message, Changed File	0,97	0,94	0,97	0,71	0,96	0,88	0,97	0,79
Added Code, Changed File	0,90	0,96	0,87	0,82	0,97	0,92	0,90	0,87
All Factors	0,97	0,97	0,98	0,88	0,96	0,92	0,97	0,90

Nightly runs, however, are advantageous in our situation. Every test case is run every night, giving us extra testing opportunities to identify any flaws that could occur with the model's predicted test case. In addition, the ML models are planned to retrained every night, reducing the risk of fault slippage more effectively. The re-training of the model takes around 3 hours on an internal cluster.

III. ANALYSIS OF RESULTS

A. Model Predictive Accuracy - RQ1

Table I shows insights about our dependent variables related to individual and combined factors and in their ability to select relevant test cases. Comparing matrices values (i.e., accuracy, precision etc.,) in Table I reveals that combining factors tends to produce better results for all matrices as compared to individual factors. Before delving into the impact of individual or combined factors on model prediction, we may infer that developers can rely on the model's ability to predict because of its increased accuracy, precision, and recall.

The precision of RF for individual or combined factors ranges between 94% and 97%, except for one combination of factors of *Added Code and Changed files*, where the precision is 87%. The same combination (*Added Code and Changed files*) resulted in a low precision of 82%, even for the NN model. This lesser precision is acceptable because changes in the source code are long strings and, due to the nature of the system under test, are often complicated, making it difficult for ML models to understand the underlying patterns. However, combining these two factors with commit messages increased precision to a suitable level (i.e., 98% and 88%), as shown in Table I, concluding that leveraging all factors in predicting failure test cases is more efficient.

The factor *Added Code* also achieved the lowest recall with RF and NN models, as 91% and 85%, respectively. The same speculation about added code being large text also applies here. On the other hand, the factor *Commit Messages* with NN and RF models, complementing the work of Yang et al. in [18], where authors produced very good results classifying commit messages by looking at the content of commit messages.

The Table I shows that combining every factor for test selection is the preferable alternative. The accuracy, precision, recall, and f1-score of all combined factors are approximately 97%. It's worth noting that NN has a precision limit of 88% for all combined factors. The reason is that this low precision is related to not utilizing the full potential of the neural network because this experiment was conducted on non-GPU equipment.

RQ 1.1, 1.2 The results show that combining factors like commit message, additional source code, and change file path has a substantial impact on predicting test cases that are more likely to fail. As an outcome of our research, Random Forest surpasses Neural Networks in terms of simplicity, interpretability, and efficacy in dealing with the unique characteristics of our dataset.

B. Model Live Evaluation - RQ2

The RQ2 investigates the gains in terms of text execution saving and test time saving. Due to SUT confidentiality, we do not display the statistics of faults occurring in the CI pipeline, as well as the average percentage of failure detection (APFD) in model predicted test cases. Such information is sensitive to organization. Not providing the APFD may create concerns for fault slippage. However, nightly runs offer an advantage in our circumstance. All test cases are conducted nightly, providing us with additional testing activities to catch the flaw, if any fault slips with the model anticipated test case.

Table II presents the gains in terms of reducing testing executions between pre-merge testing and CI loop between actual executed test cases and model predicted test case. Using a model prediction, pre-merge testing can minimize test cases by 88% by revealing most of the test failures. This allows for intelligent test case selection for both experienced developers and newcomers to the organization with limited domain knowledge. The ML model reduces test cases by 74% in the CI pipeline. The CI pipeline test case decrease is significant because the pipeline gets triggered on each merging commit, which occurs frequently throughout the day. Such test case reduction throughout the pre-merge and CI loops allows developers to receive faster feedback on their code changes. Again, there is a nightly run that executes all test cases on all commits, adding another layer of security against fault slipping.

There is no doubt that running fewer test cases takes less time, but running randomly selected test cases would provide no benefits to developers because there is no relationship between code changes and test case execution, necessitating the use of intelligent test case selection that can understand the complex relationship between code changes and which test case to execute. Table II presents the gains in terms of reducing testing time between pre-merge testing and CI loop within actual executed test cases and model predicted test case. Using a model prediction, pre-merge testing time can be minimize

TABLE II
GAINS IN TERMS OF SAVING TESTING TIME AND TEST EXECUTIONS BETWEEN PRE-MERGE AND CI TESTING AFTER COMPARING THE ACTUAL DATA WITH PREDICTED DATA

	Pre-Merged testing	CI Testing
Test Execution Saved	88%	90%
Test Time Saved	44%	74%

by 44%. The testing time during the CI loop is reduced to 74%. On an aggregation level, the 74% time over a month is a significant reduction in terms of providing faster feedback to developers after they merged their commit to master. In addition to decreasing testing time and execution, utilizing an ML model for test case selection has significant benefits for reducing stress in testing infrastructure, such as less test log storage and other artifact storage. This is the positive indirect impact of test case selection on infrastructure.

RQ 1.1, 1.2 Our findings indicate that test case selection can significantly reduce testing time and test case executions as long as the selection process understands the relationship between test cases and factors such as code modifications, commit messages, and changed file paths. This reduction in testing time and test cases will not only provide developers with faster feedback, but it will also lower the storage demand on testing infrastructure.

IV. DISCUSSION

In any large-scale organization, the test suite is expected to grow in terms of newly added test cases, either due to new feature development or to ensure that existing functionality is not disrupted, making it more difficult for test managers to maintain test suites. Running all test cases on each commit, or when developers deliver immature source code, is ineffective in terms of faster feedback and testing infrastructure. Because the privilege of running all test cases for each source change is not optimized, the challenge is to make each test execution relevant. The relevancy can be achieved by mapping each test case to the requirement or what it tests. However, in the absence of such mapping, ML models can be more useful in determining complex relationships between different variables (e.g., additional code) and which test cases to run. The important part is to identify which factor is significant.

During our experiment, we discovered that it is not advisable to rely just on test cases predicted by the ML model. There are times when a certain section of source code requires additional testing, thus dedicated test cases should run for an extended amount of time. Furthermore, an organization must consider the compliance and legal requirements that need the execution of certain tests whenever SUT changes. Similarly, new test cases should be allowed to run on each commit in order to generate enough historical data for the model to be re-trained on new test cases.

The effectiveness of test cases selected through any means (i.e., ML, history based, clustering, etc.) can be determined through many ways, such as requirements coverage, line coverage, branch coverage, similarity or diversity [7] with respect to other test cases. The ability to detect faults is one metric, but not the only one. In a scenario when the SUT has developed over time and there are few defects in the CI loop, it is preferable to employ test case selection for requirements coverage or test similarity, if available, hence maximizing the benefits of reduced testing time and test runs.

Furthermore, the selection based on fault revealing capability will become biased over time because, each time the selection is required, it will comprise only test cases that have recently failed, resulting in an insufficient number of other test cases to execute that may reveal additional unique faults.

It is not recommended to use test case selection when making critical decisions after merging a commit. Running fewer test cases always puts the product's quality at risk due to the limited number of test cases executed. However, combining test case selection with nightly runs radically alters the scenario because the test reports generated from nightly runs can provide additional insights the following day. Nightly runs can complements the test case selection during the day.

During experiments, as shown in Table I, NN performed lower than RF. This reduced performance is not due to a lack of NN capabilities, but to experiments being done on conventional CPU clusters rather than GPU clusters. As a result, when dealing with massive amounts of data and advanced machine learning algorithms, high performance computing is essential.

V. RELATED WORK

In the realm of software engineering, the application of Natural Language Processing (NLP) techniques for the analysis of commit messages provides valuable information about the software history. An influential work that is closely related to our work is from Yang et al. [18]. This paper discusses the use of natural language processing (NLP) and machine learning (ML) to study commit messages in the High Energy Physics (HEP) software development context. This research contributes to the field by proposing a ground-truth database and a machine learning prediction model that automatically identifies areas of code more prone to changes. The model achieved high performance metrics, with an average accuracy of 0.9590, precision of 0.9448, recall of 0.9382, and F1-score of 0.9360.

The other similar work [22] using code changes presents a novel approach to test prioritization in the context of continuous integration (CI) testing. The authors address the challenge of efficiently selecting test cases for regression testing in CI, where a large number of code changes trigger the execution of numerous test cases. The proposed approach leverages machine learning, specifically neural network classification, to predict the effectiveness of test cases in detecting faults for unseen code changes. The research builds upon prior work in the field of test prioritization, where techniques such as regression test prioritization (RTP) have been employed to

improve the efficiency of regression testing in CI environments. While RTP has shown promise in enhancing fault detection, the scalability of traditional RTP approaches has been a concern, particularly as the volume of historical test execution data increases. Our work, to best of our knowledge, provides the use of combination of input factors to select test cases leveraging machine learning algorithems.

VI. Conclusion

This study investigates the predictive accuracy of utilizing ML models (e.g., random forest and neural network) in selecting test cases based on source code changes to provide developers with faster feedback throughout various regression testing activities. The current scenario under study involves developers running all test suites, even for minor code changes, which increases feedback time and load on testing infrastructure. This article employed a variety of input factors, including commit message, new source code, and change file path, to select test cases from a previously trained machine learning model. The ML models were trained on 11 million test observation collected from a 15 month time period. The research paper found that combining input factors improves accuracy, precision, recall, and f1-score by approximately 97%. Random forest performs better than neural networks in terms of predicting accuracy. Furthermore, during the live evaluation of ML models in real-time testing environment, we have saved between 88% - 90% test executions and 44% - 74% testing time in different testing activities (e.g., pre-merge and CI loop).

References

[1] P. Rodríguez, A. Haghighatkhah, L. E. Lwakatare, S. Teppola, T. Suomalainen, J. Eskeli, T. Karvonen, P. Kuvaja, J. M. Verner, and M. Oivo, "Continuous deployment of software intensive products and services: A systematic mapping study," *Journal of Systems and Software*, vol. 123, pp. 263–291, Jan. 2017. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0164121215002812

[2] M. Leppänen, S. Mäkinen, M. Pagels, V. P. Eloranta, J. Itkonen, M. V. Mäntylä, and T. Männistö, "The highways and country roads to continuous deployment," *IEEE Software*, vol. 32, no. 2, pp. 64–72, Mar. 2015.

[3] M. Hilton, N. Nelson, T. Tunnell, D. Marinov, and D. Dig, "Trade-offs in continuous integration: assurance, security, and flexibility." ACM Press, 2017, pp. 197–207. [Online]. Available: http://dl.acm.org/citation.cfm?doid=3106237.3106270

[4] A. Ahmad, O. Leifler, and K. Sandahl, "The 36th ACM/SIGAPP Symposium on Applied Computing, March 22–26, 2021}{Virtual Event}," Republic of Korea, Mar. 2021.

[5] ——, "Software professionals' information needs in continuous integration and delivery," in *Proceedings of the 36th Annual ACM Symposium on Applied Computing*, ser. SAC '21. New York, NY, USA: Association for Computing Machinery, Mar. 2021, pp. 1513–1520. [Online]. Available: https://doi.org/10.1145/3412841.3442026

[6] A. Memon, Z. Gao, B. Nguyen, S. Dhanda, E. Nickell, R. Siemborski, and J. Micco, "Taming Google-scale continuous testing," in *2017 IEEE/ACM 39th International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP)*, May 2017, pp. 233–242.

[7] F. G. de Oliveira Neto, A. Ahmad, O. Leifler, K. Sandahl, and E. Enoiu, "Improving Continuous Integration with Similarity-based Test Case Selection," in *Proceedings of the 13th International Workshop on Automation of Software Test*, ser. AST '18. New York, NY, USA: ACM, 2018, pp. 39–45. [Online]. Available: http://doi.acm.org/10.1145/3194733.3194744

[8] P. E. Strandberg, W. Afzal, and D. Sundmark, "Software test results exploration and visualization with continuous integration and nightly testing," *International Journal on Software Tools for Technology Transfer*, vol. 24, no. 2, pp. 261–285, Apr. 2022. [Online]. Available: https://doi.org/10.1007/s10009-022-00647-1

[9] M. Shahin, M. A. Babar, and L. Zhu, "Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices," *IEEE Access*, vol. 5, pp. 3909–3943, 2017.

[10] J. Anderson, S. Salem, and H. Do, "Improving the Effectiveness of Test Suite Through Mining Historical Data," in *Proceedings of the 11th Working Conference on Mining Software Repositories*, ser. MSR 2014. New York, NY, USA: ACM, 2014, pp. 142–151. [Online]. Available: http://doi.acm.org/10.1145/2597073.2597084

[11] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: a survey," *Software Testing Verification and Reliability*, vol. 22, no. 2, pp. 67–120, 2012.

[12] J.-M. Kim and A. Porter, "A history-based test prioritization technique for regression testing in resource constrained environments," in *Proceedings of the 24th International Conference on Software Engineering*, ser. ICSE '02. New York, NY, USA: Association for Computing Machinery, May 2002, pp. 119–129. [Online]. Available: https://doi.org/10.1145/581339.581357

[13] T. Bach, A. Andrzejak, and R. Pannemans, "Coverage-Based Reduction of Test Execution Time: Lessons from a Very Large Industrial Project." IEEE, Mar. 2017, pp. 3–12. [Online]. Available: http://ieeexplore.ieee.org/document/7899023/

[14] S. Shamshiri, R. Just, J. M. Rojas, G. Fraser, P. McMinn, and A. Arcuri, "Do Automatically Generated Unit Tests Find Real Faults? An Empirical Study of Effectiveness and Challenges (T)." IEEE, Nov. 2015, pp. 201–211. [Online]. Available: http://ieeexplore.ieee.org/document/7372009/

[15] A. Ahmad, "Contributions to improving feedback and trust in automated testing and continuous integration and delivery," Ph.D. dissertation, Linköping University, Software and Systems, Faculty of Science & Engineering, 2022.

[16] J. H. Kwon and I. Y. Ko, "Cost-Effective Regression Testing Using Bloom Filters in Continuous Integration Development Environments," in *2017 24th Asia-Pacific Software Engineering Conference (APSEC)*, Dec. 2017, pp. 160–168.

[17] A. Vescan, R. Găceanu, and A. Szederjesi-Dragomir, "Neural network-based test case prioritization in continuous integration," pp. 68–77, 2023.

[18] Y. Yang, E. Ronchieri, and M. Canaparo, "Natural language processing application on commit messages: A case study on hep software," *Applied Sciences*, vol. 12, no. 21, 2022. [Online]. Available: https://www.mdpi.com/2076-3417/12/21/10773

[19] K. W. Al-Sabbagh, M. Staron, R. Hebig, and W. Meding, "Predicting test case verdicts using textual analysis of committed code churns," in *Joint Proceedings of the International Workshop on Software Measurement and the International Conference on Software Process and Product Measurement (IWSM Mensura 2019), Haarlem, The Netherlands, October 7-9, 2019*, ser. CEUR Workshop Proceedings, A. K. Tarhan and A. Coskunçay, Eds., vol. 2476. CEUR-WS.org, 2019, pp. 138–153. [Online]. Available: https://ceur-ws.org/Vol-2476/paper7.pdf

[20] Y. Feng, Q. Shi, X. Gao, J. Wan, C. Fang, and Z. Chen, "Deepgini: prioritizing massive tests to enhance the robustness of deep neural networks," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, ser. ISSTA 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 177–188. [Online]. Available: https://doi.org/10.1145/3395363.3397357

[21] E. G. Dada, J. S. Bassi, H. Chiroma, S. M. Abdulhamid, A. O. Adetunmbi, and O. E. Ajibuwa, "Machine learning for email spam filtering: review, approaches and open research problems," *Heliyon*, vol. 5, no. 6, p. e01802, Jun. 2019. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S2405844018353404

[22] D. Marijan, A. Gotlieb, and A. Sapkota, "Neural network classification for improving continuous regression testing," in *2020 IEEE International Conference On Artificial Intelligence Testing (AITest)*, 2020, pp. 123–124.

[23] R. Lachmann, "12.4 - machine learning-driven test case prioritization approaches for black-box software testing," 2018. [Online]. Available: https://api.semanticscholar.org/CorpusID:69392118

[24] M. Bagherzadeh, N. Kahani, and L. Briand, "Reinforcement learning for test case prioritization," *IEEE Transactions on Software Engineering*, vol. 48, no. 8, pp. 2836–2856, 2022.